Department of Computer Science and Engineering
University of Dhaka

CSE 4113: Internet Programming Lab Project

DUCSU eVote

Team Work Plan

Division of work, phases and team rules for a 4-member build

3 November 2026

Contents

1 Overview **2**

2 Ownership map **2**

3 Tech stack and repo structure **3**

4 Phase 0 — Kickoff (Days 1–3, all four) **4**

5 Phase 1 — Working election (Weeks 1–4) **6**

6 Phase 2 — Trust and hardening (Weeks 5–8) **8**

7 Dependencies and handoffs **10**

8 Git workflow and conflict rules **11**

9 Ballot secrecy rules (everyone) **12**

10 Definition of done and phase checklists **13**

11 Open decisions **14**

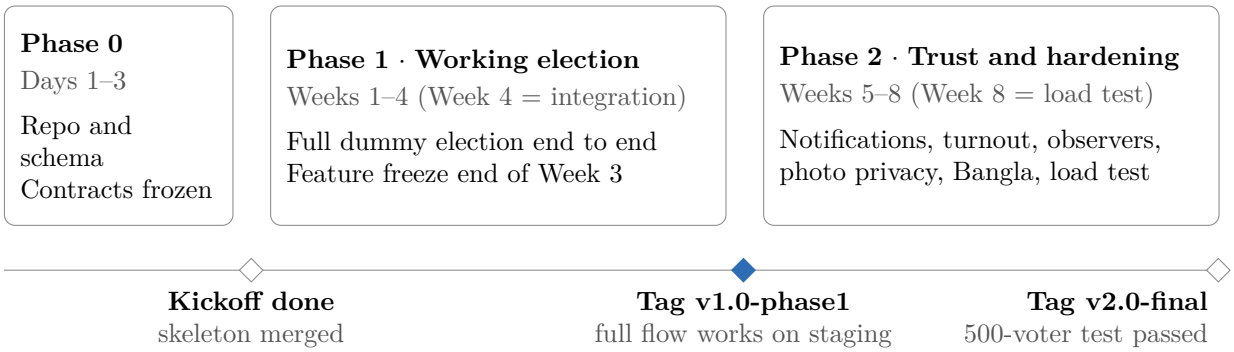
1. Overview

Each member owns one vertical slice of DUCSU eVote, from database table to screen, so two people rarely edit the same file. The build runs in two phases: Phase 1 delivers a complete election on dummy data (setup → nomination → OTP login → vote → receipt → results), and Phase 2 adds the trust, transparency and hardening features the survey asked for.

How to use this plan:

- Once roles are assigned, find your slot (M1–M4) in the ownership map. The folders, tables and API prefixes listed there are yours; anything else you change goes through a small PR reviewed by its owner.
- Days 1–3 are Phase 0: all four agree the database schema, API conventions and shared helpers. No feature code is merged before Phase 0 is merged.
- At the end of Phase 1, everyone merges to `main`, the team tests the full flow together, and tags `v1.0-phase1`. Phase 2 branches start from that tag.
- Weeks are relative. Replace them with real dates once the submission deadline is confirmed (see open decisions).

Everyone merges and tags Phase 1 before Phase 2 starts



Phase 2 branches start only from the `v1.0-phase1` tag, so unfinished Phase 1 work never mixes with Phase 2 work.

2. Ownership map

The split is by feature, not by frontend/backend, so each person can finish and test their own slice alone. Workload is balanced so each slot owns roughly 4–5 requirements plus one cross-cutting duty. Slots are not yet assigned to members.

Slot	Area	Requirements owned	Cross-cutting duty
M1	Platform, auth and notifications	FR-01, FR-02, FR-17, FR-18 (language switch), NFR-04, NFR-05, NFR-10	Repo skeleton, shared middleware, deployment, CI; reviews PRs touching shared files
M2	Election Commission and audit	FR-03, FR-04, FR-06, FR-14 (calculation), FR-16	Audit-log helper every member calls; election-status helper

Slot	Area	Requirements owned	Cross-cutting duty
M3	Candidates, panels and public pages	FR-05, FR-07, FR-13, FR-14 (results page), FR-15, FR-18 (transparency page)	UI kit and mobile layout (NFR-01, NFR-08); translation review
M4	Voting engine	FR-08, FR-09, FR-10, FR-11, FR-12, FR-19	Secrecy guardian (NFR-06); automated tests and load test (NFR-03, NFR-09)

File and route ownership:

Slot	Backend folder	API prefixes	Frontend folder	Tables owned
M1	`server/src/modules/auth`, `server/src/modules/notify`, `server/src/core/*`	`/api/auth/*`, `/api/me`	`client/src/features/auth`, `client/src/app/*`, `client/src/i18n/*` (setup only)	`admin_users`, `sessions`, `otp_codes`
M2	`server/src/modules/election`, `server/src/modules/audit`, `server/src/modules/results`	`/api/admin/elections/*`, `/api/admin/voters/*`, `/api/admin/nominations/*`, `/api/audit/*`, `/api/results/*`	`client/src/features/admin`	`elections`, `positions`, `voters`, `audit_log`, `results`
M3	`server/src/modules/candidates`, `server/src/modules/turnout`	`/api/candidates/*`, `/api/panels/*`, `/api/nominations/*`, `/api/turnout/*`, `/api/observer/*`	`client/src/features/candidates`, `client/src/features/public`, `client/src/features/observer`, `client/src/components/ui`	`candidates`, `panels`, `panel_issues`, `files`
M4	`server/src/modules/voting`	`/api/ballot/*`, `/api/vote`, `/api/receipts/*`, `/api/admin/photos/*`	`client/src/features/voting`, `client/src/features/receipts`	`ballots`, `ballot_choices`, `receipts`, `voter_photos`

When assigning slots, give each person a whole row rather than splitting a row; a row is designed to be one person's end-to-end slice.

3. Tech stack and repo structure

One monorepo, one deployed service: Express serves the API and the built React app from the same origin. That avoids CORS and cross-site cookie problems (Safari blocks HTTP-only cookies across different domains such as `vercel.app` and `onrender.com`).

Layer	Choice	Why
Frontend	React + Vite, React Router, Tailwind CSS, react-i18next, Recharts	Fast mobile-first UI, Bangla/English switch, result charts
Backend	Node.js 20 + Express, zod for validation	Matches the feasibility study; zod gives server-side validation (NFR-05)
Database	PostgreSQL with `pg` (raw parameterised SQL) and `node-pg-migrate`	Explicit `BEGIN ... SELECT ... FOR UPDATE ... COMMIT` for FR-10 is simplest in raw SQL; one migration file per change avoids a shared schema file
Auth	Server-side `sessions` table, HTTP-only Secure SameSite=Lax cookie, bcrypt	Logout and inactivity timeout are trivial with server sessions (FR-02)

Layer	Choice	Why
Email	Nodemailer over a free SMTP tier (Brevo or Gmail app password); console transport in dev	FR-01, FR-17
Jobs	`node-cron` inside the server	Reminders, photo auto-deletion
Tests	Vitest + Supertest; k6 for load	NFR-03, NFR-09
Hosting	Render or Railway (app) + Neon or Supabase (Postgres)	Free tiers; see the ephemeral-disk note below

Free hosts wipe the local disk on restart, so every upload (candidate photos, panel logos, voter photos) is stored in Postgres as `bytea`. At prototype scale (a few hundred photos of about 50 KB each) this is well within free database limits.

```
ducsu-evote/
|-- client/                     # React app
|   `-- src/
|       |-- app/                # M1: router root, layout, auth context, api client
|       |-- components/ui/      # M3: Button, Card, Modal, Stepper, FormField...
|       |-- i18n/               # M1 setup; each feature adds its own namespace files
|       |   |-- en/auth.json  en/admin.json  en/candidates.json  en/voting.json ...
|       |   `-- bn/auth.json  bn/admin.json  bn/candidates.json  bn/voting.json ...
|       `-- features/
|           |-- auth/           # M1
|           |-- admin/          # M2 (EC dashboard)
|           |-- candidates/     # M3
|           |-- public/         # M3 (home, results, transparency page)
|           |-- observer/       # M3
|           |-- voting/         # M4
|           `-- receipts/       # M4
|-- server/
|   |-- migrations/             # one timestamped file per change, never edited after merge
|   |-- seeds/                  # 01_admins (M1), 02_election (M2), 03_candidates (M3), 04_voters (M2)
|   |-- tests/                  # each module's tests live next to its owner's folder name
|   `-- src/
|       |-- core/               # M1: db.js, config.js, logger.js, middleware/, errors.js
|       |-- modules/
|       |   |-- auth/  notify/                # M1
|       |   |-- election/  audit/  results/   # M2
|       |   |-- candidates/  turnout/         # M3
|       |   `-- voting/                       # M4
|       `-- routes.js           # M1: mounts each module's router - one line per module
|-- docs/api.md                 # API contract, one section per member
|-- .github/CODEOWNERS
`-- .env.example
```

Each module exports one router and mounts it under its own prefix in `routes.js`. After Phase 0, `routes.js` should only change when a whole new module is added.

4. Phase 0 — Kickoff (Days 1–3, all four)

Phase 0 freezes the three things that cause most team conflicts: the database schema, the API shapes and the shared helpers. Do it in one 2-hour meeting on Day 1, then each person merges their part by Day 3.

Day-by-day

Day	M1	M2	M3	M4
Day 1	Create repo, `main` branch protection, CODEOWNERS; lead schema meeting	Bring election, voter and audit tables to the meeting	Bring candidate, panel and file tables	Bring ballot, receipt and photo tables; check every table against the secrecy rules
Day 2	Merge server + client skeleton, `core/` (db, config, logger, error handler), migration `001_initial_schema`	Write `docs/api.md` section for M2 routes; seed `02_election` (28 posts), `04_voters` (200 dummy voters, 5 halls)	Write `docs/api.md` section for M3 routes; seed `03_candidates` (2 panels + independents)	Write `docs/api.md` section for M4 routes; write the vote-transaction design note (steps of FR-10)
Day 3	Merge `requireAuth`, `requireRole`, `validate`, frontend `api` client, `useAuth`, i18n setup; first deploy to staging	Merge `audit.record()` and `election` status helpers (can be simple first versions)	Merge UI kit basics (Button, Card, Modal, Stepper, FormField, PageLayout) at 360 px	Merge ballot page skeleton reading seeded positions and candidates

Candidates are voters: a candidate logs in with the same registration number + OTP, and the `Candidate` role means "this voter has a row in `candidates`". Only EC and Observer accounts use email + password.

Draft schema (migration 001)

Agree this on Day 1. After merge, any change is a new migration file named by its owner, never an edit to 001.

Table	Key columns	Owner	Secrecy note
`admin_users`	id, email, password_hash, role (`ec` / `observer`), name	M1	—
`sessions`	id (random token hash), subject_type, subject_id, role, expires_at, last_seen_at	M1	—
`otp_codes`	id, voter_id, code_hash, expires_at, failed_attempts	M1	Row deleted on use or expiry
`voters`	id, reg_no (unique), name, email, hall, department, eligible, has_voted, otp_cooldown_until	M2	`has_voted` is a boolean; no `voted_at` column
`elections`	id, title, starts_at, ends_at, results_published_at, dispute_period_ends_at	M2	—
`positions`	id, election_id, name_en, name_bn, display_order, max_selections	M2	—
`panels`	id, election_id, name, slogan, logo_file_id, manifesto, created_by	M3	—
`panel_issues`	id, panel_id, issue_key, stance	M3	Used for side-by-side comparison
`candidates`	id, voter_id, position_id, panel_id (nullable), photo_file_id, bio, manifesto, status, rejection_reason	M3	—
`files`	id (uuid), mime, data (bytea)	M3	Candidate photos and panel logos only
`ballots`	id (uuid), election_id	M4	No voter id, no timestamp, no serial
`ballot_choices`	ballot_id, position_id, candidate_id	M4	—
`receipts`	code (unique), election_id	M4	No link to voter or ballot
`voter_photos`	id (uuid), voter_id, mime, data (bytea)	M4	No timestamp; never linked to ballots
`audit_log`	id, actor_type, actor_id, action, target, ip, created_at, prev_hash, hash	M2	Insert-only

Table	Key columns	Owner	Secrecy note
`results`	election_id, position_id, candidate_id, votes, is_winner, tie_flag	M2	Written only after voting closes

API conventions

- JSON in camelCase; database columns in snake_case; the repository layer converts.
- Success: `{ "data": ... }`. Error: `{ "error": { "code": "ALREADY_VOTED", "message": "..." } }` with a proper HTTP status. Error codes are listed per module in `docs/api.md`.
- All times stored and sent in UTC ISO 8601; the client shows Asia/Dhaka time.
- Every route declares its zod schema and role guard in the router file, so a reviewer sees access rules at a glance.

Shared helpers (frozen signatures)

Helper	Owner	Signature	Used by
Auth guard	M1	`requireAuth`, `requireRole('ec' | 'observer' | 'voter' | 'candidate')`; sets `req.user = { type, id, role }`	All
Validation	M1	`validate({ body, params, query })` with zod schemas	All
Mailer	M1	`notify.send({ to, template, data, lang })`	M2, M4
Audit	M2	`audit.record(req, action, target)` — reads actor and IP from `req`	All admin actions
Election status	M2	`election.getCurrent()`, `election.status()` → `draft | nomination | open | closed | published`, `election.assertEditable()` throws once voting starts	M3, M4
Frontend auth	M1	`useAuth()` → `{ user, role, login, logout }`, `<RequireRole roles={[...]}>`	All
UI kit	M3	`components/ui/*`	All

5. Phase 1 — Working election (Weeks 1–4)

Phase 1 is done when one person can run a full dummy election on staging: EC creates it, candidates nominate and get approved, voters log in with OTP and vote, receipts are issued, and results are calculated after close. Weeks 1–3 are building; Week 4 is integration and bug fixing only — no new features.

M1: platform and auth

☐ **OTP request and verify (FR-01).** `POST /api/auth/otp/request` takes a registration number, looks up the voter's university email, stores only a bcrypt/SHA-256 hash of a 6-digit code with a 5-minute expiry, and emails it. `POST /api/auth/otp/verify` accepts the code once, deletes the row, and starts a session. After 5 failed attempts, set `otp_cooldown_until` (15 minutes, configurable). Ineligible voters get the same neutral message as unknown numbers.

☐ **Admin login (FR-02).** Email + password login for EC and Observer, bcrypt hashing, seed `01_admins` with one EC and one observer account.

- ☐ **Sessions and RBAC (FR-02).** Random session token in an HTTP-only, Secure, SameSite=Lax cookie; server stores only its hash. Logout deletes the session; 15 minutes of inactivity expires it. `requireRole` on every route; `<RequireRole>` on every page.
- ☐ **Mailer (FR-17, basic).** `notify.send()` with an OTP template; console transport in development so nobody needs SMTP to test.
- ☐ **Login pages.** Voter login (registration number → OTP screen with resend timer) and admin login, mobile-first, English and Bangla strings in `i18n/*/auth.json`.
- ☐ **Staging deployment (NFR-10).** One Render/Railway service + Neon/Supabase database, environment variables documented in `.env.example`, auto-deploy from `main`.
- ☐ **CI.** GitHub Action that runs lint and tests on every PR; a PR cannot merge if it fails.

Done when: all four roles can log in and out on staging, and an OTP cannot be reused or used after 5 minutes.

M2: Election Commission and audit

- ☐ **Audit log, write side (FR-16).** `audit.record()` inserts actor, action, target, IP and time, and stores `hash = SHA-256(prev_hash + entry)`. Called for every admin action across the app.
- ☐ **Election setup (FR-03).** EC creates an election (title, start, end) and its 28 positions with display order and maximum selections. Status is computed from the clock, so voting opens and closes automatically without a job. Status banner data via `GET /api/elections/current`.
- ☐ **Edit lock (FR-04).** `election.assertEditable()` blocks changes to positions, candidates and the voter list once `starts_at` has passed; M3 calls it in candidate routes.
- ☐ **Voter list import (FR-04).** CSV upload (registration number, name, email, hall, department), row-by-row validation report (duplicates, bad emails), and eligible/ineligible toggle with search.
- ☐ **Nomination review (FR-06).** EC queue of pending nominations with photo, bio and manifesto; approve, or reject with a required reason. Each decision is audited.
- ☐ **Result calculation, first version (FR-14).** `POST /api/results/calculate` only after `ends_at`: count `ballot_choices` per candidate, mark winners up to each position's seat count, flag ties. Stores into `results`.
- ☐ **EC dashboard shell.** Sidebar with Election, Positions, Voters, Nominations, Results, Audit Log.

Done when: EC can set up the seeded election from the UI, import a CSV, approve candidates, and calculate results after close.

M3: candidates, panels and public pages

- ☐ **UI kit and layout.** Shared components and the mobile page layout at 360 px width, with WCAG AA colours chosen from the start (NFR-01, NFR-08).
- ☐ **Nomination form (FR-05).** A logged-in voter picks one position, uploads a photo (JPEG/PNG, max 2 MB, re-encoded server-side), writes bio and manifesto. One nomination per student. Candidate sees status and rejection reason on their dashboard.

- ☐ **Panels (FR-05).** Create a panel (name, slogan, logo, joint manifesto), join an existing panel, or stand as independent. Panel creator manages members.
- ☐ **Candidate browsing (FR-07, basic).** Public list of approved candidates grouped by position in display order; candidate profile page at `/candidates/:id`. Only approved candidates are visible (FR-06).
- ☐ **Candidate profile component.** A `<CandidateProfile id>` component that M4 can open inside a modal on the ballot.
- ☐ **Home page.** Election title, status banner and links to candidates, login and (later) results.

Done when: a student can nominate, join a panel, get approved by EC, and appear on the public candidate list.

M4: voting engine

- ☐ **Ballot API.** `GET /api/ballot` returns positions in order with approved candidates; only for eligible voters who have not voted, while voting is open.
- ☐ **Step-by-step ballot (FR-08).** One position per screen with a progress indicator ("Post 4 of 28"), maximum selections enforced in the browser, Skip allowed, and a profile modal (M3's component) that keeps selections. Selections live only in React state, never in localStorage.
- ☐ **Review and confirm (FR-09).** Review screen listing every position and choice (or "left blank"), edit links back to each step, and a final confirm dialog.
- ☐ **Photo capture, basic (FR-19).** Camera permission, live preview, capture to a JPEG via canvas; submit is disabled until a photo exists.
- ☐ **Cast-vote transaction (FR-10, FR-11).** `POST /api/vote` (multipart: choices + photo) runs in one transaction: lock the voter row with `SELECT ... FOR UPDATE`; check eligible, not voted, election open; validate choices against positions and maximums again on the server; insert ballot (random UUID) and choices; insert voter photo; set `has_voted = true`; insert receipt; commit. Any error rolls back everything and returns a retry message.
- ☐ **Receipt (FR-12).** Random code such as `DU-7KQ4-M9XP` from a cryptographic generator; shown with Copy and Download (.txt) buttons.
- ☐ **Double-vote test.** Automated test that fires 10 simultaneous votes for one voter and asserts exactly one ballot, one receipt and one photo exist.

Done when: a seeded voter can complete all 28 posts on a phone, gets a receipt, and a second attempt is rejected with `ALREADY_VOTED`.

6. Phase 2 — Trust and hardening (Weeks 5–8)

Phase 2 starts from tag `v1.0-phase1` and adds what makes the system trustworthy: notifications, live turnout, observers, audit integrity, public receipts, photo privacy, Bangla, and proof that it survives 500 voters. Weeks 5–7 are building; Week 8 is the load test, bug fixing and demo rehearsal.

M1: notifications, security and operations

- ☐ **Notifications (FR-17).** Emails when voting opens, a reminder to voters with `has_voted = false` (default 3 hours before close), and a "results published" email to everyone. Sent in batches with a delay to stay inside free SMTP limits. Templates in English and Bangla.
- ☐ **Security hardening (NFR-05).** `helmet` headers, HTTPS-only cookies in production, rate limits on OTP request (per registration number and per IP) and on all logins, request size limits, secrets only in environment variables, and a written security checklist in `docs/security.md`.
- ☐ **Configurable settings (NFR-09).** OTP expiry, cooldown, rate limits, reminder time and session timeout read from environment variables with defaults, documented in `.env.example`.
- ☐ **Language switch (FR-18).** Bangla/English toggle in the header, remembered per browser, applied to every voter-facing page; collect each owner's `bn/*.json` and check for missing keys with a script.
- ☐ **Backups and availability (NFR-04).** `npm run backup` (pg_dump) run before voting opens and after it closes; health endpoint `/api/health` and a free uptime monitor during the demo window. Keep the service awake during voting (free tiers sleep after inactivity).
- ☐ **Production deploy (NFR-10).** Final public URL, production seed with dummy data only, README with setup and demo accounts.

M2: results, ties and audit integrity

- ☐ **Tie resolution (FR-14).** EC screen listing flagged ties; EC picks the outcome with a written note, recorded in the audit log.
- ☐ **Publish results (FR-14).** EC publishes results; `GET /api/results` becomes public only after publishing and includes the consistency check: number of ballots = number of receipts = number of voters marked as voted.
- ☐ **Audit integrity check (FR-16).** `GET /api/audit/verify` walks the hash chain and reports the first modified or missing entry. Add a database trigger that blocks `UPDATE` and `DELETE` on `audit_log`. EC and Observer see the log with filters.
- ☐ **Audit coverage review.** Go through every admin route in all modules and confirm each one calls `audit.record()`; open issues for any gaps in other members' modules rather than editing their code.
- ☐ **EC dashboard polish.** Election timeline view, voter list counts by hall, nomination statistics.
- ☐ **Tests.** Results calculation (winners, seat counts, ties, blank positions) and audit tamper detection.

M3: turnout, observers and transparency

- ☐ **Live turnout (FR-13).** `GET /api/turnout` returns registered voters, votes cast, turnout % and turnout by hall, computed from `voters` only. Public page refreshes every 10 seconds. No candidate counts exist anywhere while voting is open.
- ☐ **Observer dashboard (FR-15).** Read-only pages for turnout, system status (election status, server health) and the audit log with M2's integrity check. No personal voter data on any observer route.

- ☐ **Results page (FR-14).** Public page with one Recharts bar chart per position, winners highlighted, tie notes, and the ballots/receipts/voted consistency line from M2's API.
- ☐ **Search, filter and panel comparison (FR-07).** Search candidates by name, filter by position, panel or hall; compare two panels side by side, issue by issue, using `panel_issues`.
- ☐ **Transparency content (FR-18).** Plain-language "How your vote stays secret" page in English and Bangla, the note on the voting page explaining why candidate counts are hidden until close, and a link to the public GitHub repository.
- ☐ **Accessibility pass (NFR-08).** Contrast check on every page, visible focus states, keyboard navigation for shared components, tested on Chrome, Firefox, Safari and Edge.

M4: photo privacy, receipts and proof

- ☐ **Photo verification, complete (FR-19, NFR-06).** Notice before the ballot explaining that a photo will be taken and why; clear messages when camera permission is denied; server re-encodes every photo with `sharp` to strip metadata; random UUID ids; no timestamp stored.
- ☐ **Dispute viewer (FR-19).** `GET /api/admin/photos/:voterId` for EC only, shown in an EC page that M2 links from the voter list; every view calls `audit.record(req, 'PHOTO_VIEWED', voterId)`.
- ☐ **Photo auto-deletion (FR-19).** Daily `node-cron` job deletes all voter photos after `dispute_period_ends_at`; audited as a system action.
- ☐ **Public receipt list (FR-12).** After voting closes, `GET /api/receipts` publishes all codes sorted alphabetically, with a search box page so voters can find theirs.
- ☐ **Load test (NFR-03, NFR-02).** k6 script that logs in and votes as 500 simultaneous seeded voters against staging; report zero errors, zero lost votes, submission time, and equal ballot/receipt/voted counts. Results saved in `docs/load-test.md`.
- ☐ **Secrecy and voting tests (NFR-09).** Tests that ballots have no link columns, that `/api/vote` bodies never reach logs, that limits are enforced server-side, and the Phase 1 concurrent double-vote test in CI.
- ☐ **Usability check (NFR-01, NFR-08).** Time three classmates completing the 28-post ballot on a phone (target under 5 minutes) and confirm the whole ballot works by keyboard.
- ☐ **Optional: encrypted ballots (FR-11 extension).** Only if everything else is done by Week 7: encrypt choices with the EC public key at cast time; M2 decrypts with the private key during calculation.

7. Dependencies and handoffs

Nobody waits on anybody: every dependency has a stub or seed data to work against until the real thing is merged. If a handoff slips, the provider says so in the group chat the same day.

Who needs it	What	From	Needed by	Work against until then
Everyone	Skeleton, migration 001, `requireAuth`, `requireRole`	M1	Day 3	Nothing — Phase 0 blocks feature work
Everyone	Seed data: 28 posts, 200 voters, 2 panels, approved candidates	M2, M3	Day 3	—

Who needs it	What	From	Needed by	Work against until then
M2, M3, M4	`audit.record()`, `election.status()`, `assertEditable()`	M2	Day 3	First version may log to console and read status from seed dates
M2, M3, M4	Real login and session cookie	M1	End of Week 1	Dev-only header `x-dev-user: voter:12` accepted when `NODE_ENV=development`
M4	`<CandidateProfile>` component	M3	End of Week 2	Simple modal showing name and photo from the ballot API
M3 (observer)	Audit log read API + verify	M2	Week 6	Mock JSON from `docs/api.md`
M3 (results page)	Published results API with consistency counts	M2	Week 6	Mock JSON from `docs/api.md`
M2 (calculation)	`ballot_choices` filled by real votes	M4	Week 3	Seed script `05_dummy_ballots` (dev only)
M1 (reminders)	List of voters who have not voted	M2	Week 5	Query `voters` directly via an M2 helper
M4 (dispute viewer link)	Link from EC voter list	M2	Week 6	Direct URL
All (Bangla)	Each owner's `bn/*.json` strings	Each owner	Week 7	English fallback

The dev-only header must be removed by the production config check in M1's Phase 2 hardening task; CI fails if it is enabled outside development.

8. Git workflow and conflict rules

Short-lived branches, small PRs and strict file ownership keep merge conflicts rare. The rules below apply to all four members in both phases.

Branches and PRs

1. `main` is protected: no direct pushes, CI must pass, one approval required.
2. Branch name: `m<number>/<short-task>`, for example `m4/vote-transaction` or `m2/csv-import`.
3. One task from this plan = one branch = one PR. Aim for under 400 changed lines; merge within 2–3 days.
4. Before opening a PR: `git fetch` and `git rebase origin/main`, run `npm test` and `npm run lint`.
5. PR title: `[M3] FR-05 nomination form`. The description lists the FR/NFR ids covered and how to test it.
6. Squash-merge, then delete the branch. Never merge someone else's PR without their approval.
7. Reviewer rotation: M1 reviews M2, M2 reviews M3, M3 reviews M4, M4 reviews M1. PRs touching shared files also need M1.

File ownership

- `.github/CODEOWNERS` maps each folder in the ownership map to its member, so GitHub requests the right reviewer automatically.
- Need a change in someone else's module? Open an issue or ask them; don't edit it in your branch.
- Shared files (`core/*`, `routes.js`, `app/*`, root `package.json`): small separate PRs only, reviewed by M1.
- Dependencies: install inside `client/` or `server/` only; mention new packages in the PR description. If two PRs both touch `package-lock.json`, the second rebases and runs `npm install` again rather than hand-merging the lockfile.

Database changes

- Never edit a merged migration. Create a new one: `npx node-pg-migrate create m3-add-panel-issue-order`.
- Only change tables you own. A column you need in another member's table is requested in an issue.
- Each member keeps their own seed file; never add rows to someone else's seed.

Translations and API docs

- Strings go in your own namespace files (`i18n/en/voting.json`, `i18n/bn/voting.json`), never one shared file.
- Update your own section of `docs/api.md` in the same PR as the route change.

Phase boundaries

- End of Week 3: Phase 1 feature freeze. Week 4 accepts bug-fix PRs only.
- End of Week 4: all four demo the full flow together on staging, then M1 tags `v1.0-phase1`.
- End of Week 7: Phase 2 feature freeze. Week 8: load test, fixes, demo rehearsal, tag `v2.0-final`.

Team rhythm

- 10-minute daily check-in in the group chat: done yesterday, doing today, blocked by.
- 30-minute sync once a week to review this plan's checkboxes and move tasks if someone is behind.

9. Ballot secrecy rules (everyone)

The core promise is that no stored data links a voter to a ballot (NFR-06), and one careless column or log line breaks it. M4 checks every PR against this list; any member can block a PR that breaks it.

1. No `created_at`, `updated_at`, serial id or sequence on `ballots`, `ballot_choices`, `receipts` or `voter_photos`. Watch for helpers or templates that add timestamps automatically.
2. No `voted_at` anywhere. `voters.has_voted` is a boolean only.
3. No foreign key or shared value between `voters`/`voter_photos` and `ballots`/`receipts`.
4. Never log the body of `/api/vote`, the receipt code, or ballot choices. The request logger skips bodies for `/api/vote` and `/api/receipts`; error logs on that route record only the error code.
5. Never read ballots or receipts in insertion order: always `ORDER BY id` (random UUID) for ballots and `ORDER BY code` for the published receipt list.
6. Ballot selections stay in React state only — not in localStorage, sessionStorage, URLs or analytics.
7. No candidate vote counts are computed or returned before `ends_at`; the results route checks the clock itself, not only the UI.
8. The audit log records administrative actions only, never a voter casting a vote.
9. Voter photos: re-encoded without metadata, stored without a time, visible to EC only, every view audited, deleted after the dispute period.
10. Demo and tests use dummy data only; no real registration numbers or emails in seeds, screenshots or the public repository.

10. Definition of done and phase checklists

A task is done only when it is merged to `main`, works on staging, and meets every point below.

Every task:

- Works at 360 px width and on desktop.
- Server validates all input with zod; role guard on every route.
- At least one automated test for the main success path and one failure path.
- English strings in i18n files (Bangla by Week 7 at the latest).
- Its `docs/api.md` section is updated.
- Admin actions call `audit.record()`.
- Passes the secrecy rules above.

Phase 1 demo checklist (end of Week 4)

☐ EC logs in, creates the election with 28 positions, imports the voter CSV.

☐ A student nominates, creates or joins a panel; EC rejects one nomination with a reason and approves another.

☐ Positions, candidates and voter list are locked after the start time.

☐ A voter logs in with OTP, completes the step-by-step ballot, reviews, takes a photo and submits.

☐ The voter gets a receipt; a second attempt is rejected; the double-vote test passes in CI.

- ☐ After the end time, EC calculates results; winners and ties appear correctly.
- ☐ Ballots, receipts and voters marked as voted are equal.
- ☐ Everything above works on the staging URL. Tag `v1.0-phase1`.

Phase 2 demo checklist (end of Week 8)

- ☐ Voting-open, reminder and results emails arrive.
- ☐ Live turnout updates every 10 seconds with hall breakdown; no candidate counts before close.
- ☐ Observer sees turnout, status and audit log, and the integrity check catches a manually edited row.
- ☐ Results page shows charts per position and the consistency line; receipt search works.
- ☐ EC views a voter photo for a dispute and the view appears in the audit log; photos are deleted after the dispute period.
- ☐ Panel comparison, search and filter work; Bangla/English switch covers all voter pages; transparency page and GitHub link are live.
- ☐ k6 run with 500 simultaneous voters: no errors, no lost votes, results saved in `docs/load-test.md`.
- ☐ Security checklist complete; WCAG contrast and keyboard checks done on four browsers.
- ☐ Production URL works with dummy data. Tag `v2.0-final`.

11. Open decisions

Confirm these at the Day 1 meeting; the plan uses the defaults shown.

- ☐ **Deadlines.** Real dates for the end of Phase 1 and Phase 2 (default: 4 weeks each).
- ☐ **Role assignment.** Decide which member takes M1, M2, M3 and M4, and note it in the ownership map.
- ☐ **Hosting.** Render vs Railway for the app; Neon vs Supabase for Postgres.
- ☐ **Email provider.** Brevo free tier vs a Gmail app password; check daily send limits against the reminder batch size.
- ☐ **Dispute period length.** Default 7 days after results are published before photos are deleted.
- ☐ **Encrypted ballots.** Attempt the FR-11 extension in Week 7, or drop it.